StreamCast Frontend

Complete Placement, Viva & Project Presentation Guide

Angular 17 · TypeScript · HTML · CSS

For: Afrin Banua | Project: streamcast-ui | Date: 19 May 2026

14 sections · Concepts · File-by-file · 100+ interview Q&A · PPT content

Table of Contents

1. Project Overview

1.1 Project Title

StreamCast — Digital Content Distribution Console (Frontend)

1.2 Project Purpose

StreamCast is an internal back-office web application used by a streaming/broadcasting company (think Netflix, Hotstar, Prime Video) to manage everything that happens *before* a movie or show reaches the viewer:

- Cataloguing every film/show + its video/audio/subtitle assets.
- Recording licensing contracts and the legal clauses inside them.
- Managing distribution partners (broadcasters, downstream streamers).
- Packaging content into delivery manifests and shipping to partners.
- Scheduling broadcast windows and resolving overlap conflicts.
- Reporting on views and revenue.
- Administering users and role-based access.

1.3 Problem Statement

Streaming businesses juggle thousands of titles, dozens of legal contracts per title, multiple delivery partners, and tight broadcast windows. Doing this in spreadsheets is error-prone — overlapping rights, missed deliveries, expired contracts, and conflicting schedules. **StreamCast centralises the workflow into one secure, role-based console** backed by a microservices API, so each team (Rights, Distribution, Scheduling, Compliance) sees only what they need.

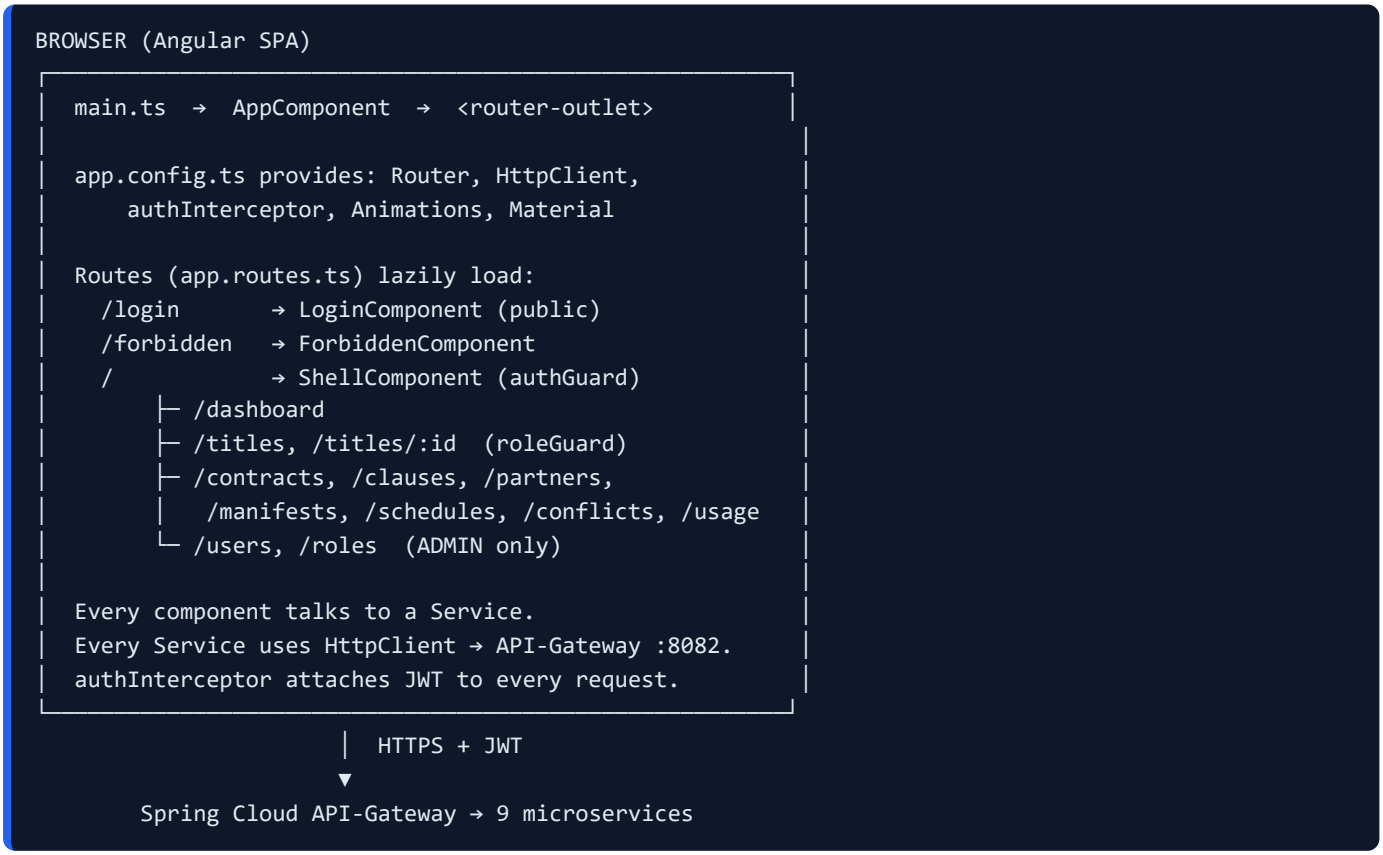
1.4 Real-World Usage

- **Content Owner** uploads a new movie → creates a Title + Assets.
- **Rights Manager** attaches a Contract granting India broadcast rights for 2 years, with exclusivity Clauses.
- **Scheduler** plans a primetime slot on Platform-X next Friday 9 PM.
- **Compliance Officer** reviews the Conflicts page if two schedules overlap.
- **Distribution Operator** creates a Manifest, ships to a Partner, tracks delivery.
- **Admin** creates users, assigns roles.

1.5 Features Implemented

Feature	Page	Allowed Roles
Login + JWT auth	/login	Everyone (public)
Dashboard with role tiles	/dashboard	All authenticated
Titles catalog (CRUD + search)	/titles	ADMIN, CONTENT_OWNER, RIGHTS_MANAGER, SCHEDULER
Title detail (assets, metadata)	/titles/:id	Same as above
Contracts	/contracts	ADMIN, RIGHTS_MANAGER, SCHEDULER, COMPLIANCE_OFFICER
Clauses	/clauses	ADMIN, RIGHTS_MANAGER, SCHEDULER, COMPLIANCE_OFFICER
Partners	/partners	ADMIN, DISTRIBUTION_OPERATOR, PARTNER_ADMIN, RIGHTS_MANAGER
Manifests (delivery packages)	/manifests	ADMIN, DISTRIBUTION_OPERATOR, PARTNER_ADMIN
Schedules (calendar)	/schedules	ADMIN, SCHEDULER, RIGHTS_MANAGER, COMPLIANCE_OFFICER
Conflicts	/conflicts	ADMIN, SCHEDULER, COMPLIANCE_OFFICER
Usage analytics	/usage	ADMIN, COMPLIANCE_OFFICER, SCHEDULER
Users admin	/users	ADMIN only
Roles admin	/roles	ADMIN only

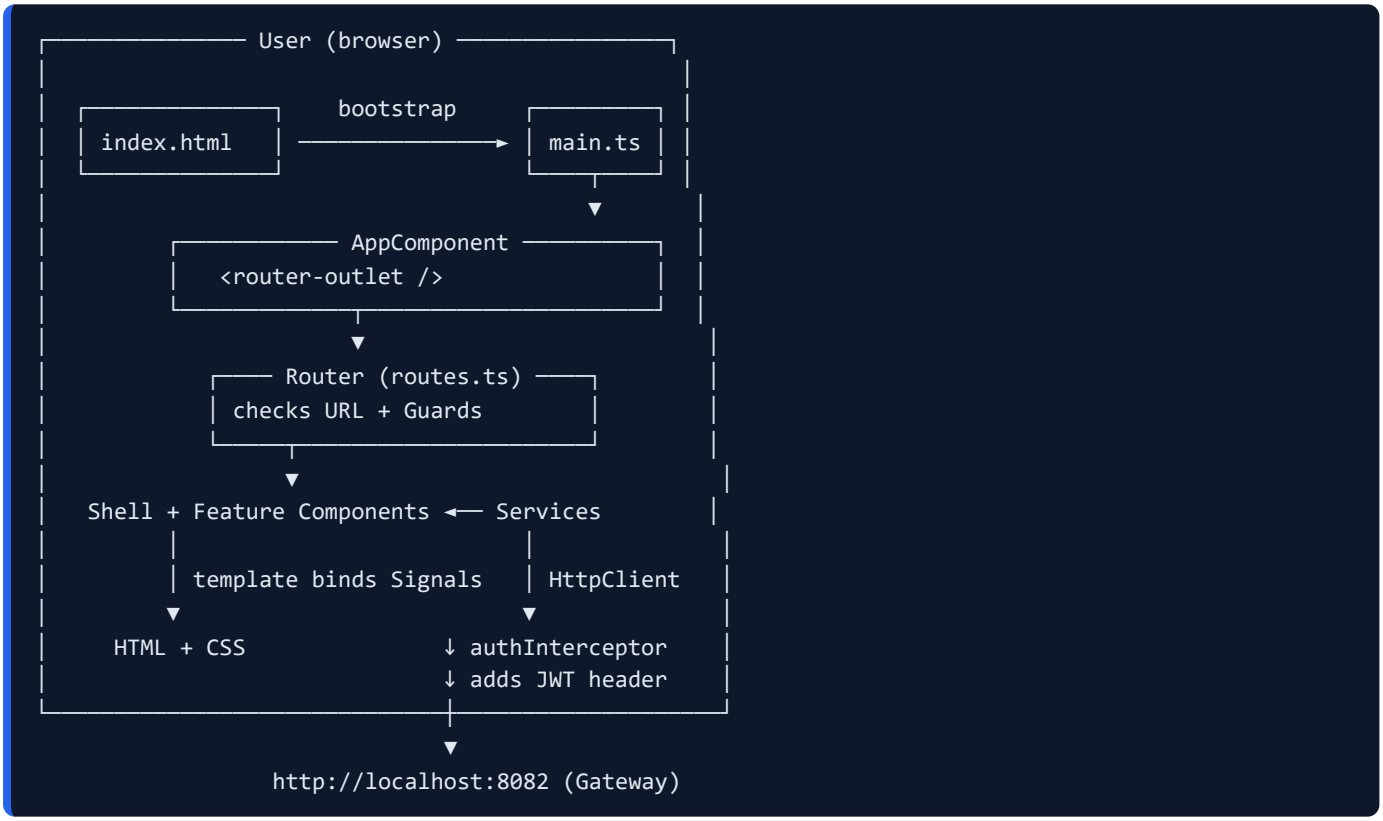
1.6 Angular Architecture Overview



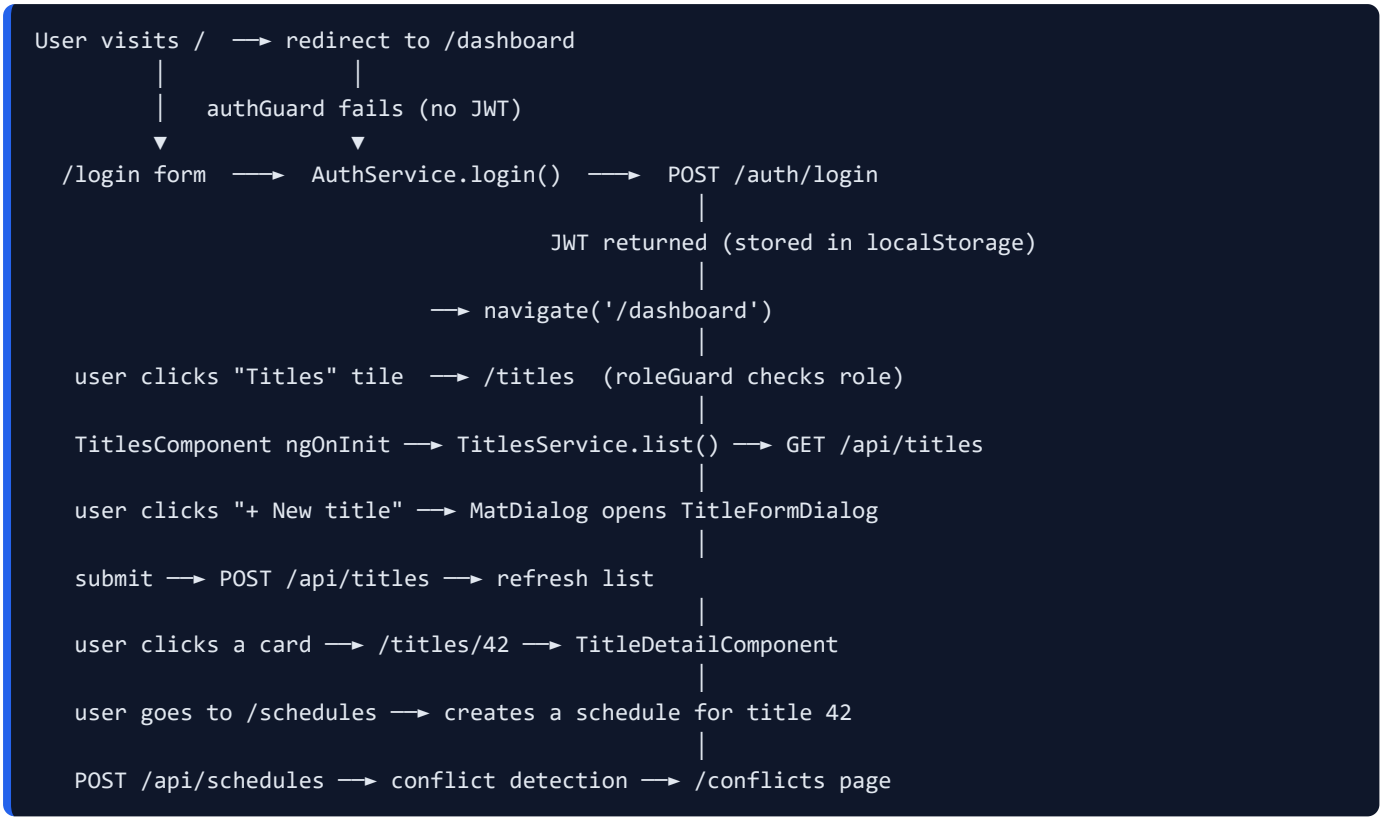
1.7 Complete Application Flow

1. `index.html` loads → bootstraps Angular via `main.ts`.
2. `AppComponent` renders `<router-outlet />`.
3. Router checks the URL. For protected routes, `authGuard` runs first.
4. If guard passes, the route's `loadComponent()` downloads the chunk on demand.
5. The component is constructed → its dependencies (services) are injected.
6. The component fetches data via its service using `HttpClient`.
7. The `authInterceptor` adds the `Authorization: Bearer …` header automatically.
8. Response arrives → service maps it → component updates a Signal → template re-renders.
9. On user action (click/submit), the component calls the service again (POST/PUT/DELETE).
10. On 401, the interceptor logs the user out and redirects to `/login`.

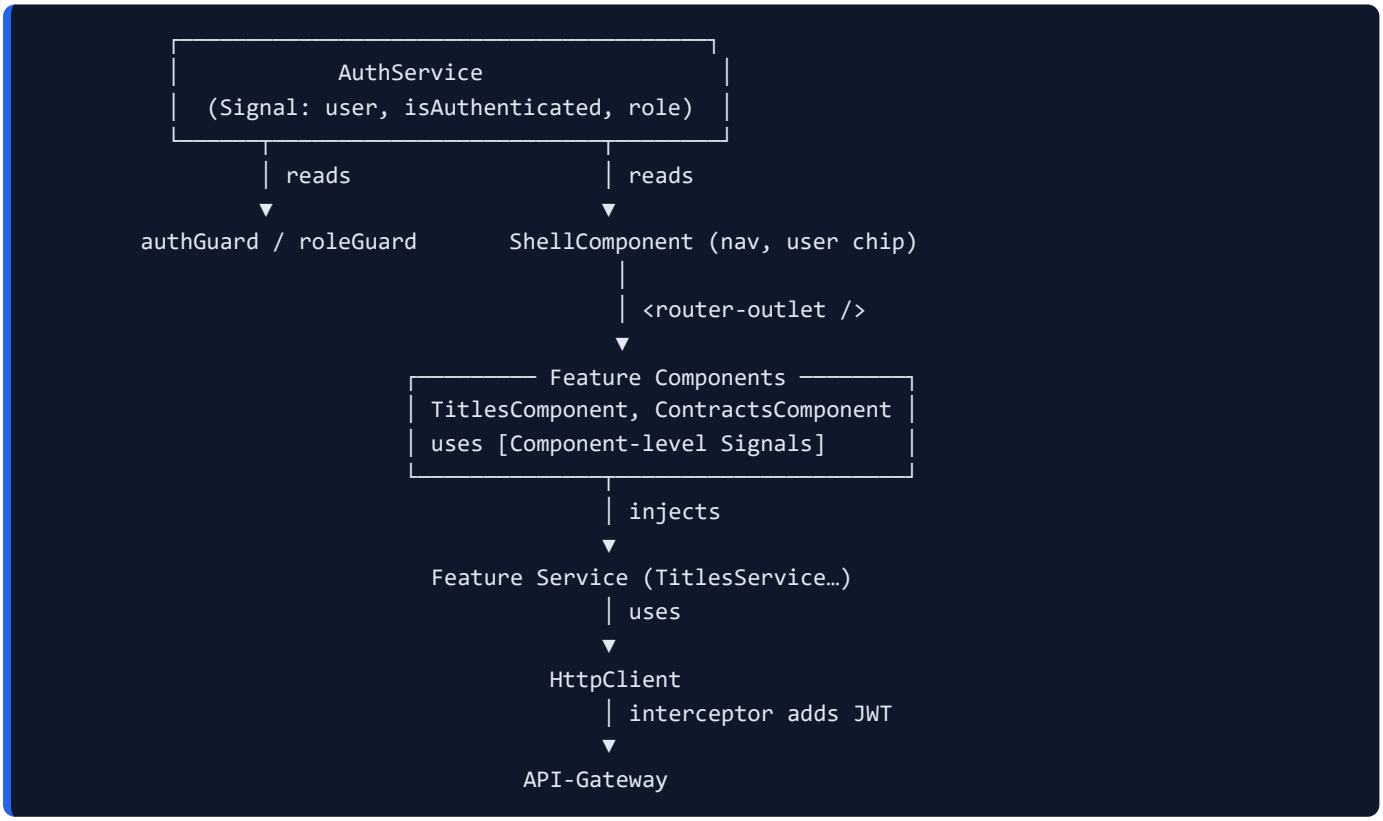
1.8 Architecture Diagram



1.9 User Flow Diagram (login → schedule a title)



1.10 Component Communication Diagram



2. Angular Concepts Used (Deep Dive)

For each concept: **What → Why → Where in project → Advantages → Real-time example → Interview Qs → Common mistakes**.

2.1 Angular Basics

What: Angular is a TypeScript-based SPA framework by Google. It organises a UI as a tree of *components*, talks to a backend via *services*, and changes screen via the *router*.

Why: Two-way binding, dependency injection, strong typing, batteries-included tooling.

Where: The whole `streamcast-ui/` folder.

Advantages: Component reuse, TypeScript safety, opinionated structure → easy to onboard, integrated forms/HTTP/router/animations.

Real-time example: Banking dashboards, Gmail, Microsoft Office Online — all built with Angular or similar frameworks.

> **Q. What is Angular?**
> A TypeScript SPA framework that uses components, services, and the router to build dynamic single-page applications.

> **Q. Difference between Angular and AngularJS?**
> AngularJS (1.x) uses JavaScript + scope. Angular (2+) is a complete rewrite using TypeScript, components, and tree-shakable Ivy compiler.

Common mistakes: Mixing AngularJS tutorials with Angular code; forgetting that Angular is component-based, not page-based.

2.2 Components

What: The fundamental UI building block. Each component = `@Component`-decorated class + HTML template + CSS.

Why: Encapsulation, reusability, testability, clear ownership.

Where: Every `*.component.ts` file — `LoginComponent`, `DashboardComponent`, `TitlesComponent`, `ShellComponent`, etc.

```
@Component({
  selector: 'sc-login',         // becomes <sc-login> tag
  standalone: true,
  imports: [CommonModule, ReactiveFormsModule],
  template: `<form>...</form>`
})
export class LoginComponent {}
```

Advantages: Replace HTML+JS spaghetti with reusable units. Material-style design systems are built on this idea.

Real-time example: One `TitleCard` rendered 50 times in a grid. Change once → all 50 update.

> **Q. What is a component?**
> A reusable UI element with its own template, style, and logic, decorated with `@Component`.

Common mistakes: Putting business logic in a component instead of a service; not declaring imports for standalone components.

2.3 Modules & Standalone Components

What: Earlier Angular versions required every component to be inside an `NgModule`. Angular 14+ introduced **standalone components** — they declare their own imports and don't need an `NgModule`.

Why standalone: Less boilerplate, faster lazy loading, simpler mental model.

Where: *Every* component in StreamCast uses `standalone: true`. There is no `AppModule`. `main.ts` uses `bootstrapApplication()` instead of `bootstrapModule()`.

```
// main.ts
bootstrapApplication(AppComponent, appConfig);

// every component
@Component({ standalone: true, imports: [...], template: ... })
```

> **Q. Standalone components vs NgModule?**
> Standalone components declare their own imports directly. NgModules grouped components, but added boilerplate. Standalone is the modern default.

2.4 TypeScript

What: JavaScript with static types added at compile time.

Why: Catches bugs before runtime; great IDE autocomplete; interfaces describe data shape.

Where: All `.ts` files. Interfaces in `core/models/` describe API data.

```
// core/models/user.ts
export type Role = 'ADMIN' | 'CONTENT_OWNER' | 'RIGHTS_MANAGER' | ...;
export interface CurrentUser { email: string; role: Role; token: string; expiresAt: number; }
```

Real-time example: Renaming a field in `Title` immediately flags every component that used the old name.

> **Q. Difference between `interface` and `type` in TS?**
> Both describe shape. `interface` can be extended/merged. `type` supports unions and primitives. Either works for objects.

2.5 HTML Templates

What: Angular templates extend HTML with bindings and directives (`{{ }}` , `[prop]` , `(event)` , `*ngIf`).

Why: Lets you describe what the UI should look like for any state, with no manual DOM manipulation.

Where: Inline in the `template:` of each component (StreamCast uses inline templates everywhere).

2.6 CSS Styling

StreamCast originally used Tailwind utility classes; the styling has been converted to **plain CSS** in two files: `src/utilities.css` (utility classes used by templates) and `src/styles.scss` (Material theme + global rules).

2.7 Data Binding (All Four Forms)

Type	Syntax	Direction	Example from StreamCast
Interpolation	`{{ value }}`	TS → DOM	`<div>{{ t.name }}</div>`
Property binding	`[prop]="expr"`	TS → DOM	`[disabled]="form.invalid"`
Event binding	`(event)="handler()"`	DOM → TS	`(click)="openCreate()"`
Two-way binding	`[(ngModel)]="x"`	Both	Used in template forms (reactive forms preferred here)

> **Q. What is two-way binding?**
> Property + event in one syntax: `[(ngModel)]="x"` = `[ngModel]="x" (ngModelChange)="x=$event"` .

2.8 Directives — `ngIf` , `ngFor` , `ngClass` , `ngStyle`

```
<div *ngIf="loading()">Spinner</div>
<li *ngFor="let t of filtered(); trackBy: trackId">{{ t.name }}</li>
<span [ngClass]="{ 'sc-chip-success': t.status === 'AVAILABLE' }">{{ t.status }}</span>
<div [ngStyle]="{ background: poster(t) }"></div>
```

- **ngIf** — adds/removes element from DOM.
- **ngFor** — loops over an array; `trackBy` tells Angular which row is which for efficient updates.
- **ngClass** — adds/removes CSS classes conditionally.
- **ngStyle** — sets inline styles dynamically.

> **Q. Difference between `*ngIf` and `[hidden]` ?**
> `*ngIf` removes from DOM (cheaper if rarely shown). `[hidden]` keeps it but sets `display:none` (cheaper to toggle often).

2.9 Pipes

What: Format data in templates without changing the underlying value: `{{ value | pipe:args }}`.

Where in StreamCast: `DatePipe` formats release dates — `{{ t.releaseDate | date:'mediumDate' }}`. `CurrencyPipe` / `NumberPipe` could format revenue.

> **Q. Pure vs impure pipes?**
> Pure pipes (default) re-run only when the input reference changes. Impure pipes run on every change-detection cycle — use sparingly.

2.10 Services

What: Plain TypeScript classes that hold logic / talk to the backend, decorated with `@Injectable`.

Why: Keep components focused on UI; logic is reusable; testable in isolation.

Where: `AuthService`, `TitlesService`, `ContractsService`, `SchedulesService`, etc.

```
@Injectable({ providedIn: 'root' })
export class TitlesService {
  private http = inject(HttpClient);
  list(): Observable<Title[]> { return this.http.get<Title[]>(...); }
}
```

`providedIn: 'root'` makes the service a *singleton* — only one instance per app, available to inject anywhere.

> **Q. Why use services?**
> Single Responsibility — keep API calls and shared state out of components. They are also singletons and easily mockable in tests.

2.11 Dependency Injection (DI)

What: Angular's mechanism for giving a class the dependencies it needs (services, router, http client) without it constructing them.

Two styles used in StreamCast:

```
// Style 1: constructor
constructor(private api: TitlesService) {}

// Style 2: inject() function (preferred in standalone code)
private api = inject(TitlesService);
```

> **Q. What is DI?**
> A design pattern where a class declares what it needs; the framework supplies it. Decouples creation from usage and makes testing easy.

2.12 Routing

What: Maps URLs to components. SPA → no full page reload.

Where: `app.routes.ts` + `provideRouter(routes)` in `app.config.ts`.

Key idea — lazy loading: each route uses `loadComponent: () => import(...)` so the JS chunk is fetched only when the user actually navigates there. Initial bundle stays small.

```
{
  path: 'titles',
  canActivate: [roleGuard(['ADMIN','CONTENT_OWNER',...])],
  loadComponent: () => import('./features/catalog/titles.component')
                       .then(m => m.TitlesComponent)
}
```

2.13 Route Parameters

The route `titles/:id` reads `id` from the URL inside the component:

```
private route = inject(ActivatedRoute);
const id = +this.route.snapshot.paramMap.get('id')!;
```

With `withComponentInputBinding()` (used in `app.config.ts`), you can also bind URL params straight to `@Input()`s.

2.14 Reactive Forms

What: Forms defined in TypeScript, not HTML.

Where: `LoginComponent`, `TitleFormDialog`, every dialog.

```
form = this.fb.nonNullable.group({
  email: ['', [Validators.required, Validators.email]],
  password: ['', Validators.required]
});
```

Template uses `[formGroup]` + `formControlName`. Read with `form.value`; check validity with `form.invalid`.

2.15 Template-driven Forms

What: Forms defined inside the template using `[(ngModel)]` + `#fname="ngModel"`. Best for tiny forms.

Where: Not used in StreamCast — the project standardised on Reactive Forms because they scale better and validation is in code.

> **Q. Reactive vs Template-driven forms?**
>
> Reactive: defined in TS, sync, typed, easy to unit-test, used for complex/large forms. Template-driven: defined in HTML with `ngModel`, async, less typing, used for tiny forms.

2.16 Form Validation

Built-in validators: `Validators.required`, `Validators.email`, `Validators.minLength`, `Validators.pattern`.

In `LoginComponent` the submit button is disabled until both fields are valid:

```
<button type="submit" [disabled]="form.invalid || loading()">Sign in</button>
```

2.17 API Integration & HttpClient

What: `HttpClient` is Angular's wrapper over `fetch`/XHR. It returns Observables.

Where: Every service injects `HttpClient`.

```
this.http.get<Title[]>(`${environment.apiBase}/api/titles`)
```

The generic `<Title[]>` tells TypeScript the response shape; runtime cast is your responsibility.

2.18 Observables

What: A lazy stream of values over time. You *subscribe* to start it; *unsubscribe* to stop.

Where: Every HTTP call returns an Observable; signals do not replace Observables for HTTP.

```
this.api.list().subscribe({
  next: data => this.rows.set(data),
  error: () => this.snack.open('Failed', 'OK')
});
```

2.19 RxJS Operators

Operator	Purpose	Used in
`map`	Transform value	Services unwrapping `ApiResponse<T>` envelope
`tap`	Side effect, no transform	`AuthService.login()` stores the JWT in `tap`
`catchError`	Handle errors	`authInterceptor` catches 401 and logs out
`switchMap`	Chain dependent calls, cancel previous	Useful in search-as-you-type

2.20 Lifecycle Hooks

- `ngOnInit` — runs once after input bindings set. Fetch initial data here.
- `ngOnChanges` — runs when any `@Input()` changes.
- `ngOnDestroy` — cleanup (unsubscribe, clear timers).
- `ngAfterViewInit` — DOM ready, child components accessible.

StreamCast often does init work inside the `constructor` (fine for tiny components like `TitlesComponent.refresh()` in `constructor()`).

2.21 Guards

What: Functions returning `true | false | UrlTree` to allow or block navigation.

Where: `authGuard` (logged in?), `roleGuard(['ADMIN', …])` (correct role?). Defined in `core/auth/auth.guard.ts`.

2.22 Interceptors

Run on every HTTP request. The `authInterceptor` in StreamCast:

- Clones the request and adds `Authorization: Bearer <token>`.
- Catches 401 errors and calls `auth.logout()` → redirect to `/login`.

2.23 Authentication & Authorization

- **Authentication** = "who are you?" → login + JWT.
- **Authorization** = "what can you do?" → roles + guards + UI `*ngIf="canEdit()"`.

2.24 LocalStorage vs SessionStorage

	localStorage	sessionStorage
Lifetime	Until cleared explicitly	Until tab/window closes
Shared across tabs	Yes (same origin)	No
Capacity	~5–10 MB	~5 MB

StreamCast uses **localStorage** with key `sc.auth` to persist the JWT through page refreshes.

2.25 Error Handling

- Service uses `{ error: ... }` callback in `subscribe`.
- Errors classified by status: 0 (network), 400 (bad data), 401/403 (auth), 500+ (server).
- UI shows `MatSnackBar` for transient errors and inline messages for forms.

2.26 Responsive Design

Achieved via CSS classes with breakpoints (`sm:` 640px, `md:` 768px, `lg:` 1024px, `2xl:` 1536px). Example from titles grid:

```
class="grid grid-cols-2 sm:grid-cols-3 lg:grid-cols-4 2xl:grid-cols-6 gap-4"
```

Phones see 2 columns; tablets 3; laptops 4; ultra-wide 6.

3. Folder Structure

```
streamcast-ui/
├─ angular.json              ← workspace + build config
├─ package.json              ← npm dependencies + scripts
├─ tsconfig.json             ← TypeScript compiler options
├─ tsconfig.app.json         ← app-specific TS overrides
└─ src/
   ├─ index.html             ← single HTML entry point
   ├─ main.ts                ← bootstraps Angular
   ├─ styles.scss            ← global Material theme + base
   ├─ utilities.css          ← all CSS utility classes (was Tailwind)
   ├─ environments/
   │   └─ environment.ts     ← apiBase URL etc.
   └─ app/
      ├─ app.component.ts    ← root component (just <router-outlet>)
      ├─ app.config.ts       ← global providers
      ├─ app.routes.ts       ← all routes + guards
      ├─ core/               ← cross-cutting reusable code
      │   ├─ auth/
      │   │   ├─ auth.service.ts
      │   │   ├─ auth.guard.ts
      │   │   ├─ auth.interceptor.ts
      │   │   └─ jwt.utils.ts
      │   └─ models/
      │       ├─ user.ts, catalog.ts, contract.ts,
      │       ├─ clause.ts, partner.ts, manifest.ts,
      │       ├─ schedule.ts, usage.ts, admin.ts
      ├─ layout/
      │   └─ shell.component.ts   ← top bar, side nav, outlet
      └─ features/
          ├─ auth/login.component.ts
          ├─ dashboard/dashboard.component.ts
          ├─ catalog/{titles, title-detail}.component.ts
          ├─ catalog/{titles, assets}.service.ts
          ├─ contracts/{contracts.component.ts, contracts.service.ts}
          ├─ clauses/{clauses.component.ts, clauses.service.ts}
          ├─ partners/{partners.component.ts, partners.service.ts}
          ├─ manifests/{manifests.component.ts, manifests.service.ts}
          ├─ schedules/{schedules.component.ts, schedules.service.ts}
          ├─ conflicts/conflicts.component.ts
          ├─ usage/{usage.component.ts, usage.service.ts}
          ├─ admin/{users, roles}.component.ts + admin.service.ts
          └─ misc/forbidden.component.ts
```

Why this structure?

- **core/** = singleton things used everywhere (auth, models).
- **layout/** = chrome around feature pages (header, side nav).
- **features/** = one folder per business capability. Each holds its own components + service. Easy to find. Easy to delete a feature without leaks.
- **environments/** = swap config (dev vs prod) at build time.
- **assets/** = static images / fonts / icons (here: `logo.svg`).

Each folder's purpose at a glance

Folder	Purpose
`src/`	All source code Angular needs.
`app/`	All application code.
`core/auth/`	Auth service, JWT helper, guards, HTTP interceptor.
`core/models/`	TypeScript interfaces shared by components & services.
`layout/`	Outer shell (nav + outlet).
`features/X/`	One feature: component(s) + service + maybe sub-components.
`environments/`	Build-time configuration (API URL, production flag).

4. File-by-File Explanation

4.1 `src/index.html`

Purpose: The one HTML page the browser loads. Angular replaces `<sc-root></sc-root>` with the app.

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8" />
  <title>Streamcast Console</title>
  <base href="/" />            <!-- router uses this -->
  <link rel="icon" type="image/svg+xml" href="/assets/logo.svg" />
  <link href="https://fonts.googleapis.com/css2?family=Inter..." rel="stylesheet"/>
  <link href="https://fonts.googleapis.com/icon?family=Material+Icons" rel="stylesheet"/>
</head>
<body class="mat-typography bg-bg text-ink-100">
  <sc-root></sc-root>
</body>
</html>
```

- `<base href="/" />` — required by Angular Router to resolve URLs.
- `Inter` font + `Material Icons` loaded from Google CDN.
- `<sc-root>` matches the selector of `AppComponent`.

4.2 `src/main.ts`

```
import { bootstrapApplication } from '@angular/platform-browser';
import { AppComponent } from './app/app.component';
import { appConfig } from './app/app.config';

bootstrapApplication(AppComponent, appConfig).catch(err => console.error(err));
```

Step-by-step:

1. Imports Angular's standalone bootstrap function.
2. Imports the root component (the one selector matched in index.html).
3. Imports the app-wide providers config.
4. Calls `bootstrapApplication` — Angular renders `AppComponent` into `<sc-root>` and registers all providers.

If you delete this file: Angular has no entry point — nothing renders.

4.3 `app/app.component.ts`

```
@Component({
  selector: 'sc-root',
  standalone: true,
  imports: [RouterOutlet],
  template: `<router-outlet />`
})
export class AppComponent {}
```

Purpose: Pure shell — its only template is `<router-outlet />` where the router renders whatever component matches the URL.

4.4 `app/app.config.ts`

```
export const appConfig: ApplicationConfig = {
  providers: [
    provideRouter(routes, withComponentInputBinding()),
    provideHttpClient(withInterceptors([authInterceptor])),
    provideAnimations(),
    importProvidersFrom(MatSnackBarModule)
  ]
};
```

Line-by-line:

- `provideRouter(routes, withComponentInputBinding())` — installs router + lets URL params bind directly to `@Input()` s.
- `provideHttpClient(withInterceptors([authInterceptor]))` — installs `HttpClient` and chains the JWT interceptor on every request.
- `provideAnimations()` — needed by Angular Material for dialogs, ripples, etc.
- `importProvidersFrom(MatSnackBarModule)` — older-style module imported into the standalone world for global snack-bars.

4.5 `app/app.routes.ts`

Defines every URL the app understands. Highlights:

```
{ path: '', pathMatch: 'full', redirectTo: 'dashboard' },          // home → dashboard
{ path: 'login', loadComponent: () => import('...').then(...) },   // public, lazy
{
  path: '',
  canActivate: [authGuard],                                        // every child needs login
  loadComponent: () => import('./layout/shell.component')...,      // shell wraps them
  children: [
    { path: 'dashboard', loadComponent: ... },
    { path: 'titles',
      canActivate: [roleGuard(['ADMIN','CONTENT_OWNER','RIGHTS_MANAGER','SCHEDULER'])],
      loadComponent: ... },
    ...
  ]
},
{ path: '**', redirectTo: 'dashboard' }                            // wildcard 404
```

Why lazy loading? Each `loadComponent` downloads only when that route is visited. Initial bundle < 250 KB; the rest is fetched on demand.

4.6 `core/auth/auth.service.ts`

```
@Injectable({ providedIn: 'root' })
export class AuthService {
  private http = inject(HttpClient);
  private router = inject(Router);
  private readonly _user = signal<CurrentUser | null>(this.restore());
  readonly user = this._user.asReadonly();
  readonly isAuthenticated = computed(() => {
    const u = this._user(); return !!u && u.expiresAt > Date.now();
  });
  readonly role = computed<Role | null>(() => this._user()?.role ?? null);

  login(req: LoginRequest): Observable<string> {
    return this.http.post(`${environment.apiBase}/auth/login`, req,
                          { responseType: 'text' })
                    .pipe(tap(token => this.persist(token)));
  }
  logout(): void { localStorage.removeItem(STORAGE_KEY); this._user.set(null);
this.router.navigate(['/login']); }
  hasAnyRole(roles: Role[]): boolean { const r = this._user()?.role; return !!r &&
roles.includes(r); }
  private persist(token: string): void {
    const claims = decodeJwt(token);
    const user: CurrentUser = { email: claims.sub, role: claims.role as Role, token, expiresAt:
claims.exp * 1000 };
    localStorage.setItem(STORAGE_KEY, JSON.stringify(user));
    this._user.set(user);
  }
  private restore(): CurrentUser | null { /* read localStorage, expire check */ }
}
```

Key ideas:

- `signal(...)` + `computed(...)` = Angular 17 reactivity. Anyone reading `auth.isAuthenticated()` re-renders when the user logs in/out.
- `tap` in the login observable persists the token without changing the value passed downstream.
- `restore()` auto-logs in the user on page refresh if their JWT is still valid.

4.7 `core/auth/auth.guard.ts`

```
export const authGuard: CanActivateFn = () => {
  const auth = inject(AuthService); const router = inject(Router);
  if (auth.isAuthenticated()) return true;
  router.navigate(['/login']); return false;
};

export const roleGuard = (roles: Role[]): CanActivateFn => () => {
  const auth = inject(AuthService); const router = inject(Router);
  if (!auth.isAuthenticated()) { router.navigate(['/login']); return false; }
  if (auth.hasAnyRole(roles)) return true;
  router.navigate(['/forbidden']); return false;
};
```

What: Functional guards. `authGuard` blocks unauthenticated users; `roleGuard` is a factory — pass the allowed roles, get a guard.

4.8 `core/auth/auth.interceptor.ts`

```
export const authInterceptor: HttpInterceptorFn = (req, next) => {
  const auth = inject(AuthService); const router = inject(Router);
  const token = auth.user()?.token;
  const cloned = token ? req.clone({ setHeaders: { Authorization: `Bearer ${token}` } }) : req;
  return next(cloned).pipe(
    catchError(err => {
      if (err?.status === 401) { auth.logout(); router.navigate(['/login']); }
      return throwError(() => err);
    })
  );
};
```

Runs for every HTTP call. Adds the JWT and reacts to 401s globally.

4.9 `core/auth/jwt.utils.ts`

```
export function decodeJwt(token: string): JwtClaims | null {
  try {
    const payload = token.split('.')[1];
    const json = atob(payload.replace(/-/g, '+').replace(/_/g, '/'));
    return JSON.parse(json) as JwtClaims;
  } catch { return null; }
}
```

Decodes the base64 payload (no signature verification — that's the server's job). Used to read `sub` (email), `role`, `exp`.

4.10 `core/models/*` (TypeScript interfaces)

File	Defines
user.ts	`Role` union, `CurrentUser`, `LoginRequest`
catalog.ts	`Title`, `Asset`, `Metadata`
contract.ts	`Contract`, `ApiResponse<T>`
clause.ts	`Clause`
partner.ts	`Partner`
manifest.ts	`Manifest`
schedule.ts	`Schedule`, `Conflict`, `CalendarSchedule`, `ScheduleApiResponse<T>`
usage.ts	`UsageRecord`, `UsageSummary`, `UsageBreakdown`
admin.ts	`UserDef`, `UserCreate`, `RoleDef`

4.11 `environments/environment.ts`

```
export const environment = { production: false, apiBase: 'http://localhost:8082' };
```

Single source of truth for the gateway URL. `environment.prod.ts` would point to the production URL.

4.12 `src/styles.scss`

Global styles + Angular Material dark theme. Loads `Inter` font, sets body bg `#08090d`, defines custom scrollbar, overrides Material dialog/menu/tab colours for a dark UI.

4.13 `src/utilities.css`

~44 KB of plain CSS containing every utility class used by templates (`.flex`, `.bg-bg`, `.text-ink-100`, `.sc-card`, `.sc-btn-primary`, responsive variants, etc.). Generated by compiling Tailwind once and freezing the output.

4.14 `layout/shell.component.ts`

The visual frame around every authenticated page:

- Top bar with logo, breadcrumb, user avatar dropdown.

- Left side nav with role-filtered links (computed from `auth.role()`).
- `<router-outlet />` in the centre that renders the current page.
- `logout()` → calls `AuthService.logout()`.

4.15 `features/auth/login.component.ts`

Reactive form with email + password, password show/hide toggle, inline error banner, loading spinner.

```
form = this.fb.nonNullable.group({
  email: ['', [Validators.required, Validators.email]],
  password: ['', Validators.required]
});

submit(): void {
  if (this.form.invalid || this.loading()) return;
  this.loading.set(true);
  this.auth.login(this.form.getRawValue()).subscribe({
    next: () => { this.loading.set(false); this.router.navigate(['/dashboard']); },
    error: err => { /* maps status → friendly message */ }
  });
}
```

The error mapper distinguishes status 0 (gateway down), 400 (bad request), 401/403 (bad creds), 5xx (server) — gives the user actionable feedback.

4.16 `features/dashboard/dashboard.component.ts`

Reads `auth.role()` and renders coloured tiles for every feature the user is allowed to access. Each tile is a `routerLink` to that feature.

4.17 `features/catalog/titles.component.ts` + `titles.service.ts`

Service: Plain CRUD wrapper over `/api/titles`.

Component:

- Uses three signals: `rows`, `loading`, `query`.
- `filtered` is a `computed` derived from `rows()` + `query()` — re-runs only when those change.
- Grid of poster cards with `*ngFor` + `trackBy` for performance.
- "New title" button opens `TitleFormDialog` (Material Dialog) → on save calls `api.create()` then refreshes.
- Card click routes to `/titles/:id` via `router.navigate`.

4.18 `features/catalog/title-detail.component.ts`

Shows title hero header + tabs for Assets, Metadata. Uses route param `:id` to fetch the title and related resources.

4.19 Other feature components (same pattern)

File	What it does
contracts.component.ts + contracts.service.ts	List + create/edit/delete contracts. Service unwraps `ApiResponse<T>` envelope using `map`.
clauses.component.ts + clauses.service.ts	List + create clauses. Each clause references a contractId.
partners.component.ts + partners.service.ts	Partner CRUD with validation (10-digit phone, name 2–50 chars).
manifests.component.ts + manifests.service.ts	Create delivery manifests; log attempts/receipts.
schedules.component.ts + schedules.service.ts	Calendar view + create. Calls `/api/schedules/calendar?start=&end=`.
conflicts.component.ts	Lists conflicts; calls resolve endpoint.
usage.component.ts + usage.service.ts	Summary / details / breakdown reports with date-range pickers.
admin/users.component.ts + admin/roles.component.ts + admin.service.ts	ADMIN-only — CRUD over users + roles.
misc/forbidden.component.ts	Static 403 page.

4.20 Best practices reflected in the code

- **One service per feature** — small, single-purpose.
- **Strong typing everywhere** — every endpoint has a model.
- **Signals for local state**, RxJS for HTTP — right tool for the job.
- **Guards centralised in `core/auth`** — no role logic inside templates beyond `canEdit()`.
- **Interceptor handles 401 globally** — no copy-pasted error blocks.

5. HTML Explanation

5.1 Page structure (Login example)

```
<div class="relative min-h-screen flex items-center justify-center bg-bg">
  <!-- decorative background -->
  <div class="absolute inset-0 bg-grad-hero"></div>

  <!-- card -->
  <div class="sc-glass p-8">
    <form [formGroup]="form" (ngSubmit)="submit()">
      <input formControlName="email" class="sc-input" />
      <input formControlName="password" class="sc-input" />
      <button type="submit" class="sc-btn-primary" [disabled]="form.invalid">Sign in</button>
    </form>
  </div>
</div>
```

5.2 Why each HTML tag is used

Tag	Purpose
`<form>`	Logical group of inputs, allows Enter-to-submit + a single submit handler.
`<input>`	Single-line user data entry.
`<label>`	Accessible name for inputs (clickable to focus).
`<button>`	Action trigger. `type="submit"` submits the form; `type="button"` doesn't.
`<table>`	Tabular data (used in admin pages, manifests list).
`<nav>`	Semantically marks navigation regions.
`<div> / <section>`	Layout containers. `section` when content has a heading.

5.3 Angular template syntax cheat-sheet

Syntax	Meaning
`{{ x }}`	Interpolation — print value.
`[prop]="x"`	Property binding (TS → DOM).
`(event)="fn()"`	Event binding (DOM → TS).
`[(ngModel)]="x"`	Two-way binding.
`*ngIf="cond"`	Conditionally include element.
`*ngFor="let i of list"`	Repeat element per item.
`#ref`	Template reference variable.
`ng-container`	Wrap structural directives without producing extra DOM.
`ng-template #x`	Define a template fragment (used with `*ngIf else`).

5.4 User interaction flow inside a template

1. User types → input fires `input` event → form control updates → `form.valueChanges` emits.
2. Angular re-evaluates bindings (`form.invalid` changes).
3. Disabled button toggles automatically.
4. User presses Enter → `(ngSubmit)` calls the component's submit handler.

6. CSS Explanation

6.1 Styling approach

StreamCast uses two CSS layers:

1. **Global theme** (`styles.scss`) — Material dark theme + base body + scrollbar + Material overrides.
2. **Utility classes** (`utilities.css`) — pre-written single-purpose classes like `.flex` , `.gap-4` , `.bg-surface-2` , `.rounded-2xl` . Templates compose UI by listing these classes.

Component-class layer (e.g. `.sc-card` , `.sc-btn-primary` , `.sc-input` , `.sc-chip-success`) groups multiple utilities for reuse — applied as a single class name in templates.

6.2 Layout structure (Flexbox + Grid)

```
/* Side-by-side header */
.flex { display: flex; }
.items-center { align-items: center; }
.justify-between { justify-content: space-between; }
.gap-4 { gap: 1rem; }

/* Responsive grid of poster cards */
.grid { display: grid; }
.grid-cols-2 { grid-template-columns: repeat(2, minmax(0, 1fr)); }
@media (min-width: 640px) {
  .sm\:grid-cols-3 { grid-template-columns: repeat(3, minmax(0, 1fr)); }
}
@media (min-width: 1024px) {
  .lg\:grid-cols-4 { grid-template-columns: repeat(4, minmax(0, 1fr)); }
}
```

6.3 Responsive design

Mobile-first. Default = phone. Then `sm:` overrides at 640px, `md:` at 768px, `lg:` at 1024px, `2xl:` at 1536px.

6.4 Form styling

```
.sc-input {
  display: block; height: 2.75rem; width: 100%;
  border: 1px solid #272a35; border-radius: 0.5rem;
  background: #0f1015; color: #e7e9ee;
  padding: 0 0.875rem; transition: box-shadow .15s, border-color .15s;
}
.sc-input:focus {
  border-color: #f43f5e;
  box-shadow: 0 0 0 4px rgba(244,63,94,0.18);
}
```

6.5 Button styling

```
.sc-btn-primary {
  display: inline-flex; align-items: center; justify-content: center;
  height: 2.75rem; padding: 0 1rem; gap: 0.5rem;
  font-size: 14px; font-weight: 600; color: #fff;
  border-radius: 0.5rem;
  background: linear-gradient(to right, #e11d48, #f43f5e);
  transition: filter .15s, transform .15s;
}
.sc-btn-primary:hover { background: linear-gradient(to right, #be123c, #e11d48); }
.sc-btn-primary:disabled { opacity: .5; cursor: not-allowed; }
```

6.6 Card design

```
.sc-card {
  border-radius: 1rem;
  background: #15161e;
  border: 1px solid #272a35;
  box-shadow: 0 1px 0 rgba(255,255,255,.04) inset, 0 8px 24px rgba(0,0,0,.35);
}
.sc-card-hover:hover {
  transform: translateY(-2px);
  box-shadow: 0 4px 12px rgba(0,0,0,.4), 0 16px 40px rgba(0,0,0,.45);
}
```

6.7 Navigation styling

Side-nav uses a vertical flex layout with active-link highlighting via `routerLinkActive`. Active state changes background and adds a left brand-coloured indicator.

6.8 Possible UI improvements

- Use `prefers-reduced-motion` to disable animations for accessibility.

- Add a light theme via CSS custom properties.
- Make the dashboard responsive on very small phones (currently 2-col on <640px).
- Add skeleton loaders instead of spinner-only states.

7. API Flow

7.1 Big picture

7.2 GET — list resources

```
// titles.service.ts
list(): Observable<Title[]> { return this.http.get<Title[]>(this.base); }

// component
this.api.list().subscribe({
  next: rows => this.rows.set(rows),
  error: err => this.snack.open('Failed to load')
});
```

Endpoint: `GET /api/titles` → **response body:** `Title[]`.

7.3 POST — create

```
create(t: Title): Observable<Title> { return this.http.post<Title>(this.base, t); }
```

Request body: `{ name, releaseDate, genre, language, status }`. **Response:** the created Title with `id`.

7.4 PUT — update

```
update(id: number, t: Title) { return this.http.put<Title>(`${this.base}/${id}`, t); }
```

7.5 DELETE

```
delete(id: number) { return this.http.delete(`${this.base}/${id}`); }
```

7.6 HttpClient features used

Feature	Where used
Type generics `<T>`	All services
`HttpParams`	`SchedulesService.calendar(start, end)`
`responseType: 'text'`	`AuthService.login()` (JWT comes as text, not JSON)
Generic envelope mapping	`ContractsService`, `SchedulesService` use `.pipe(map(r => r.data))`

7.7 Observable lifecycle

1. **Cold** — nothing happens until you `subscribe()`.
2. On `subscribe()`, HTTP request fires.
3. Server responds → observable emits one `next` value, then `complete`.
4. If you don't unsubscribe, HttpClient observables auto-complete (no leak), unlike custom long-lived streams.

7.8 Error handling per request

```
this.api.list().subscribe({
  next: rows => this.rows.set(rows),
  error: (err: HttpErrorResponse) => {
    if (err.status === 0)        msg = 'Network error';
    else if (err.status === 404) msg = 'Not found';
    else if (err.status >= 500)  msg = 'Server error';
    else                          msg = err.error?.message ?? 'Request failed';
    this.snack.open(msg);
  }
});
```

8. Authentication Flow

8.1 End-to-end diagram

8.2 JWT contents (decoded payload)

```
{ "sub": "afrin@example.com", "role": "ADMIN", "iat": 1715000000, "exp": 1715086400 }
```

8.3 Why localStorage (and the trade-off)

- **Pro:** Survives page refresh → user stays logged in.
- **Con:** Vulnerable to XSS. *Mitigation:* never inject untrusted HTML, sanitize user content, use Content Security Policy.
- **Stronger alternative:** HttpOnly cookies + CSRF protection.

8.4 Route protection layers

1. **authGuard** — must have a non-expired JWT.
2. **roleGuard(['ADMIN', ...])** — must have one of the listed roles.
3. **Backend Spring Security** — re-verifies JWT signature + role on every request (defence in depth).

9. Component Communication

9.1 Parent → Child via `@Input()`

```
// child
export class StatusBadge {
  @Input() status: 'AVAILABLE' | 'DRAFT' | 'RETIRED' = 'DRAFT';
}

// parent template
<sc-status-badge [status]="t.status"></sc-status-badge>
```

9.2 Child → Parent via `@Output()` + `EventEmitter`

```
// child
@Output() saved = new EventEmitter<Title>();
emit() { this.saved.emit(this.value); }

// parent template
<sc-title-form (saved)="refresh($event)"></sc-title-form>
```

9.3 Sibling / unrelated components — shared service

The cleanest pattern, and the one StreamCast uses everywhere. The service holds a Signal; multiple components read/write it.

```
@Injectable({ providedIn: 'root' })
export class AuthService {
  private _user = signal<CurrentUser | null>(null);
  readonly user = this._user.asReadonly();    // anyone can read
  setUser(u: CurrentUser) { this._user.set(u); }  // only AuthService can write
}
```

`ShellComponent` reads `auth.user()` to show the avatar; `LoginComponent` writes through `auth.login()`. They never need direct references to each other.

9.4 Route-based communication

- **URL params** — `/titles/:id`.
- **Query params** — `/usage?start=&end=`.
- **Router state** — pass data via `router.navigate([...], { state: {…} })`.

9.5 Dialogs return data

```
this.dialog.open(TitleFormDialog).afterClosed()
    .subscribe(value => { if (value) this.api.create(value).subscribe(...); });
```

10. Interview Preparation — 100 Q&A

10.1 Angular Beginner (Q1–Q30)

1. What is Angular?

A TypeScript-based SPA framework by Google built around components, services, and the router.

2. What is a Single Page Application?

A web app that loads one HTML page once and updates the UI dynamically via JS, avoiding full reloads.

3. Why TypeScript?

Static types catch bugs early, give IDE autocomplete, and document data shapes via interfaces.

4. What is a component?

A reusable UI piece = decorator + class + template + style.

5. What is a selector?

The custom HTML tag the component matches, e.g. `sc-login`.

6. What is a module?

Legacy unit that groups components, directives, pipes, providers. Modern Angular uses standalone components instead.

7. Standalone component?

A component that declares its own imports and doesn't belong to any `NgModule`.

8. Four data-binding types?

Interpolation, property, event, two-way.

9. Difference between `{{ x }}` and `[innerHTML]`?

`{{ }}` escapes HTML; `[innerHTML]` renders it (and Angular sanitises).

10. `ngIf` vs `hidden`?

`ngIf` removes node from DOM; `hidden` hides via CSS.

11. `ngFor` with trackBy — why?

Tells Angular how to identify rows so it patches only the changed DOM, not re-renders the whole list.

12. What is a directive?

A class that changes element behaviour or appearance. Structural (`*ngIf`) vs attribute (`ngClass`).

13. What is a pipe?

A template-only transformer: `{{ value | date:'medium' }}` .

14. Pure vs impure pipe?

Pure runs only when input reference changes; impure runs on every change-detection cycle.

15. What is a service?

A reusable class that holds logic/data, injected wherever needed.

16. What is dependency injection?

A pattern where the framework supplies what a class needs instead of the class creating it.

17. `providedIn: 'root'` ?

Makes the service a singleton available app-wide.

18. What is routing?

Mapping URLs to components without page reloads.

19. What is lazy loading?

Loading a route's code only when the user navigates to it. `loadComponent: () => import(...)` .

20. How to read a route param?

`this.route.snapshot.paramMap.get('id')` or via `withComponentInputBinding()` .

21. Reactive forms vs template-driven?

Reactive in TS, sync, typed, scalable. Template-driven in HTML with `ngModel` , async, smaller forms.

22. How to validate a form?

Built-in `Validators.required` , `Validators.email` , etc., or a custom function.

23. What is HttpClient?

Angular's typed HTTP wrapper that returns Observables.

24. What is an Observable?

A lazy stream of values you subscribe to.

25. Observable vs Promise?

Promise = single value, eager. Observable = stream, lazy, cancellable, composable with operators.

26. What is RxJS?

A reactive programming library for working with streams using operators like `map` , `filter` , `switchMap` .

27. What is a Subject?

An Observable that is also an Observer — you can `next()` values into it manually.

28. `BehaviorSubject` vs `Subject` ?

BehaviorSubject holds a current value and emits it to new subscribers.

29. Lifecycle hooks?

`ngOnInit` , `ngOnChanges` , `ngDoCheck` , `ngAfterContentInit` , `ngAfterViewInit` , `ngOnDestroy` .

30. When to use `ngOnInit` vs constructor?

Constructor for DI; `ngOnInit` for setup that needs inputs to be set.

10.2 Angular Intermediate (Q31–Q60)

31. What is a guard?

A function returning `true|false|UrlTree` that allows or blocks navigation.

32. Types of guards?

`CanActivate` , `CanActivateChild` , `CanDeactivate` , `CanMatch` , `Resolve` .

33. What is an interceptor?

Middleware that runs on every HTTP request — useful for auth headers and error handling.

34. What is a Signal?

Angular 17 reactive primitive — a holder of a value plus a notification when it changes.

35. Signal vs Observable?

Signals are sync, pull-based, simple. Observables are async, push-based, composable.

36. `computed()` ?

A derived signal that recomputes only when its dependencies change.

37. `effect()` ?

A side-effect runner that executes whenever the signals it reads change.

38. What is change detection?

Angular's process of checking templates and updating the DOM. Default strategy = check everything; `OnPush` = check only when inputs change or events fire.

39. What is `OnPush` ?

A change-detection strategy that re-checks the component only when its inputs change by reference or events occur.

40. `@Input()` and `@Output()` ?

Component IO: input receives data from parent; output emits events to parent.

41. `@ViewChild()` ?

Reference a child component or DOM element from the template.

42. `HostListener` & `HostBinding` ?

Listen to/bind to events/properties on the host element of a directive.

43. `ng-content` ?

Slot for projected content (Angular's transclusion).

44. `ng-template` vs `ng-container` ?

`ng-template` = inert template fragment. `ng-container` = wrapper that renders no DOM.

45. `async` pipe?

Subscribes to an Observable/Promise in the template and unsubscribes on destroy.

46. `FormBuilder` ?

A helper that creates `FormGroup` / `FormControl` with less boilerplate.

47. `FormGroup` vs `FormArray` ?

FormGroup = fixed map of controls. FormArray = dynamic list of controls.

48. Cross-field validation?

A custom validator attached to the FormGroup that checks multiple controls together.

49. How to call APIs?

Inject `HttpClient` in a service; call `get/post/put/delete` ; subscribe in the component.

50. How to cancel an HTTP call?

Unsubscribe, or use operators like `switchMap` that cancel previous inner observables.

51. How to set a header globally?

Use an HTTP interceptor (see `authInterceptor`).

52. `switchMap` vs `mergeMap` ?

switchMap cancels previous inner observable on new emission. mergeMap runs all in parallel.

53. `concatMap` ?

Queues inner observables — runs the next only after the previous completes. Use for ordered writes.

54. Error handling in RxJS?

`catchError` operator returns a fallback observable; `retry` / `retryWhen` retry.

55. Difference between `tap` and `map` ?

`tap` performs a side effect without changing the value. `map` returns a transformed value.

56. How to share a single HTTP call between subscribers?

Use `shareReplay(1)` .

57. Memory leaks in Observables?

Forgetting to unsubscribe from long-lived streams. Use `async` pipe or `takeUntilDestroyed()` in Angular 16+.

58. `takeUntilDestroyed()` ?

Auto-unsubscribes when the component/directive is destroyed.

59. What is route resolver?

A function that pre-fetches data before activating a route, so the component opens with data ready.

60. `canDeactivate` guard?

Asks "are you sure you want to leave?" — useful for forms with unsaved changes.

10.3 Angular Advanced (Q61–Q80)

61. AOT vs JIT?

AOT compiles templates at build time → smaller, faster bundles. JIT compiles in the browser (legacy).

62. Tree-shaking?

Bundler removes unused exports at build time, reducing JS size.

63. What is Zone.js?

Patches async APIs (setTimeout, promises) so Angular knows when to run change detection.

64. Zoneless Angular?

Newer Angular versions can run without Zone.js, using Signals for change detection.

65. How does `HttpClient` differ from `fetch`?

HttpClient returns Observables (cancellable, composable), has interceptors, typed responses, automatic JSON parsing.

66. SSR (Angular Universal)?

Server-side rendering — renders the page on the server for first paint, then hydrates on the client.

67. Hydration?

Reuses server-rendered DOM on the client instead of throwing it away.

68. NgRx?

A Redux-pattern state library for Angular: store, actions, reducers, effects, selectors.

69. When to use NgRx?

Large apps with shared global state and many cross-cutting events; otherwise services + Signals suffice.

70. Service workers?

Browser-level proxy for offline caching; Angular has `@angular/service-worker`.

71. What is Ivy?

Angular's modern rendering engine — smaller bundles, faster builds, locality of compilation.

72. Standalone APIs?

`provideRouter`, `provideHttpClient`, `provideAnimations` — replace module-style imports.

73. `inject()` function?

Lets you DI dependencies outside of constructors (e.g. in functional guards/interceptors).

74. Standalone vs module-based testing?

Standalone tests use `TestBed.configureTestingModule({ imports: [Component] })` directly.

75. Performance tips?

OnPush, trackBy, lazy loading, avoid heavy work in templates, use Signals, virtual scrolling for huge lists.

76. Difference between `NgZone.run` and `NgZone.runOutsideAngular`?

`run` brings work back into the zone (triggers CD). `runOutsideAngular` opts out for perf-sensitive code.

77. What is Angular Material?

Google's component library implementing Material Design (buttons, dialogs, tables, snack-bars).

78. What is CDK?

Component Dev Kit — primitives (overlay, portal, drag-drop, virtual-scroll) used to build custom components.

79. What is content projection?

Slot child markup inside a component using `ng-content`.

80. Multi-slot projection?

`<ng-content select="[header]"></ng-content>` — projects only elements with attribute `header`.

10.4 HTML / CSS / TS Mixed (Q81–Q100)

81. Difference between `id` and `class`?

`id` unique per page; `class` can repeat.

82. Block vs inline element?

Block fills its parent and starts on a new line (`div`); inline flows with text (`span`).

83. `display: flex` vs `display: grid`?

Flex is 1D (row or column). Grid is 2D (rows + columns).

84. CSS specificity order?

Inline > ID > class/attr/pseudo-class > element > `*`. `!important` overrides everything.

85. `position: absolute` vs `fixed`?

Absolute is relative to nearest positioned ancestor. Fixed is relative to the viewport.

86. Box model?

content → padding → border → margin. `box-sizing: border-box` includes padding+border in width.

87. Media query example?

`@media (min-width: 768px) { ... }` applies styles when viewport ≥ 768px.

88. Why mobile-first?

Easier to add features for larger screens than remove for smaller. Smaller default = better perf on mobile.

89. CSS variable?

`--brand: #f43f5e; color: var(--brand);` — runtime-themable.

90. `let` vs `const` vs `var`?

`const` = immutable binding, block-scoped. `let` = mutable, block-scoped. `var` = function-scoped, legacy.

91. Arrow functions vs regular?

Arrow functions don't have their own `this`; they capture the enclosing scope.

92. `Promise.all` vs `Promise.race`?

`all` waits for all; `race` resolves with the first.

93. Difference between `==` and `===`?

`==` coerces types; `===` doesn't. Always use `===`.

94. What is hoisting?

JS moves variable/function declarations to the top of the scope. `var` is hoisted (undefined); `let` / `const` are in temporal dead zone.

95. What is event loop?

The mechanism that picks tasks from the queue and runs them on the single JS thread (macrotask + microtask queues).

96. `async/await` internals?

Syntactic sugar over Promises; `await` pauses the function until the Promise settles.

97. CORS?

Cross-Origin Resource Sharing — the browser blocks calls to a different origin unless the server sends `Access-Control-Allow-Origin`.

98. How does Angular handle CORS?

It doesn't — the backend (here API-Gateway) must respond with the correct CORS headers.

99. localStorage vs cookie for JWT?

localStorage = easy but XSS-readable. HttpOnly cookie = safer from XSS but needs CSRF protection.

100. Why is XSS dangerous?

An injected script can read localStorage (your JWT) and impersonate the user.

10.5 Project Viva Questions

> **"Explain your project."**
>
> "StreamCast is a web console for a streaming company to manage their content lifecycle — titles, contracts, clauses, partners, manifests, schedules, and usage reports. The frontend is built with Angular 17 standalone components, talks to a Spring Boot microservices backend via an API gateway. Authentication uses JWT stored in localStorage; an HTTP interceptor attaches the token to every request, and route guards plus role-based authorization control which pages each user can see. The UI uses Angular Material + a custom CSS utility layer for a dark, cinematic look."

> **"Challenges faced?"**
>
> "(1) Role-based UI: 8 roles with different permissions across 13 pages — solved by centralising in `authGuard` / `roleGuard` and reading `auth.role()` via Signals in templates. (2) Generic API envelopes: some services return `ApiResponse<T>` wrappers, others plain arrays — solved by unwrapping with `.pipe(map(r => r.data))` in the service so components always see the data. (3) Token expiry: handled globally in the interceptor — on 401 we log out and redirect."

> **"Why Angular?"**
>
> "Angular gives me TypeScript by default, batteries-included routing/HTTP/forms/animations, a strong CLI, and great IDE support. Compared to React, it's more opinionated — which helps a team of beginners stay consistent. Signals (Angular 17) give us simple, fine-grained reactivity without learning state libraries."

> **"Why services?"**
>
> "To keep components thin (UI only) and put logic + HTTP + state in reusable singletons. Easier to test, easier to mock, no duplication."

> **"Reactive vs Template-driven forms?"**
>
> "Reactive forms are defined in TypeScript — synchronous, typed, easier to validate and unit-test. Template-driven forms are defined in HTML using `ngModel`, simpler for tiny forms but harder to scale. StreamCast uses Reactive Forms everywhere."

> **"Explain Angular lifecycle hooks."**
>
> "They are methods Angular calls at specific points: `ngOnChanges` when inputs change, `ngOnInit` once after first inputs set (fetch data here), `ngDoCheck` on every change-detection run, `ngAfterContentInit` when projected content is ready, `ngAfterViewInit` when child views are ready, `ngOnDestroy` for cleanup."

> **"Explain Observables."**
>
> "An Observable is a lazy stream of values. Nothing happens until you subscribe. Operators like `map`, `filter`, `switchMap` let you compose async logic declaratively. Unlike a Promise, Observables can emit multiple values and be cancelled."

11. Debugging & Common Errors

11.1 Angular runtime errors

Error	Cause	Fix
`NG0100 ExpressionChangedAfterChecked`	You changed a bound value after change detection finished.	Wrap the update in `setTimeout` or move it to a lifecycle hook.
`NG0203 inject() must be called…`	Calling `inject()` outside of a DI context.	Move call into a constructor, factory, or `runInInjectionContext`.
`NG8001 Unknown element`	Component used in template but not imported.	Add to the `imports` array of the standalone component.
`NullInjectorError: No provider for X`	Service not provided / not `providedIn:'root'`.	Add `@Injectable({ providedIn:'root' })` or provide it in `app.config.ts`.

11.2 Routing issues

- **Blank page after refresh** — server isn't redirecting to `index.html` on deep links. Configure dev server / Spring Boot to do so.
- **Route param undefined** — typo in `:id` or reading before parent route resolved. Use `paramMap` not legacy `params`.
- **Guard fires twice** — usually a parent + child guard. Decide whether you need both.

11.3 API errors

Symptom	Likely cause
Status 0	Network down, gateway not running, mixed content (https→http).
Status 401	Token missing/expired. Interceptor handles this.
Status 403	Logged in but wrong role.
Status 404	Wrong URL — check `environment.apiBase` and the path.
Status 500	Backend bug — read backend logs.

11.4 CORS issues

Symptom: Browser console — "blocked by CORS policy". **Fix:** only call the gateway (`http://localhost:8082`), never call services directly; gateway emits CORS headers.

11.5 Template errors

- **Cannot read 'x' of undefined** — bind only after data arrives: `*ngIf="data() as d"`.
- **Unknown directive** — missing `CommonModule` in standalone imports.

11.6 Observable issues

- **HTTP call doesn't fire** — you didn't `subscribe()`.
- **Memory leak** — long-lived stream never unsubscribed. Use `async` pipe or `takeUntilDestroyed()`.
- **Double fire on submit** — `(click)` AND `(ngSubmit)` both firing. Use one.

11.7 DI errors

- "No provider for `HttpClient`" — you forgot `provideHttpClient()` in `app.config.ts`.
- "Circular dependency" — Service A injects Service B which injects Service A. Break the cycle by extracting a third class.

11.8 How to debug

1. Open DevTools → Console for runtime errors.
2. DevTools → Network for HTTP — check request URL, headers (Authorization!), status, response body.
3. Use `console.log` in components/services; check the browser source map shows real TS lines.
4. Install **Angular DevTools** Chrome extension — inspect component tree, profiler.
5. For Signals, log them via `effect(() => console.log(value()))`.

12. Best Practices

Clean architecture

- **Smart vs dumb components** — feature components fetch data; small reusable components only render.
- **One feature = one folder** — drop-in / drop-out without leaks.
- **Models live in** `core/models` — shared by services and components.

Reusable components

- Extract repeated UI into a child component with `@Input` / `@Output`.
- Use `ng-content` for slot-based composition.

Service-based architecture

- One service per feature; never put HTTP in components directly.
- Wrap envelopes (`ApiResponse<T>`) in the service so components see only `T`.

Naming conventions

- **files**: kebab-case → `titles.service.ts` , `title-detail.component.ts` .
- **classes**: PascalCase → `TitlesService` , `TitleDetailComponent` .
- **booleans**: `is` / `has` / `can` prefix.
- **observables**: trailing `$` (e.g. `users$`) when assigned to a property.

Performance

- **Lazy load every route** (already done in StreamCast).
- **trackBy in** `*ngFor` for big lists.
- **OnPush + Signals** for fewer change-detection passes.
- **Virtual scrolling** (CDK) for 1000+ rows.
- **Production build** with AOT, minification, sourcemaps off.

Security

- Always validate on the backend too; client checks are UX only.
- Sanitize any user content rendered as HTML (Angular sanitises by default for `[innerHTML]`).
- Never log JWTs to console in production.
- Set short JWT expiry; use refresh tokens for long sessions.
- Lock down CORS to known origins in the gateway.

13. PPT Slide Content

Copy these bullets into PowerPoint / Google Slides. Notes below each slide are speaker notes.

Slide 1 — Title

- StreamCast Console
- A Digital Content Distribution Platform
- Built with Angular 17 + Spring Boot Microservices
- Presented by Afrin Banua

Greet the panel. Mention this is the frontend (Angular) on top of a 10-service Java backend.

Slide 2 — Problem Statement

- Streaming companies juggle 1000+ titles, contracts, partners, schedules.
- Spreadsheets cause overlapping rights, missed deliveries, expired licenses.
- Need: a single, secure, role-based console.

Frame the pain: legal risk + ops chaos = need software.

Slide 3 — Technology Stack

- Frontend: **Angular 17, TypeScript, HTML, CSS, Angular Material**
- Backend: Spring Boot, Spring Cloud Gateway, Eureka, OpenFeign, Resilience4j
- Database: MySQL (one per microservice)
- Auth: JWT

Briefly justify: Angular for SPA, Spring Cloud for microservices, JWT for stateless auth.

Slide 4 — Architecture Overview

- Single-page Angular app calls Spring Cloud Gateway (:8082).
- Gateway routes to 9 microservices via Eureka discovery.
- JWT validated at the gateway; downstream services trust forwarded identity.

Show the architecture diagram. Mention defence-in-depth on the backend.

Slide 5 — Folder Structure

- core/ → auth, models, interceptor, guards
- layout/ → shell (top bar + side nav + outlet)
- features/ → one folder per business capability
- environments/ → API URLs, prod/dev flags

Emphasise feature isolation = easier to maintain.

Slide 6 — Angular Concepts Used

- Standalone components (no NgModule)
- Signals + computed for state
- Reactive Forms + Validators
- HttpClient + Observables + RxJS operators
- Functional Guards + HTTP Interceptor
- Lazy loading every feature route

Highlight modern Angular: standalone, Signals, functional guards.

Slide 7 — Routing & Guards

- `authGuard` — only logged-in users.
- `roleGuard(['ADMIN', …])` — only allowed roles.
- Forbidden role → /forbidden.
- 13 routes, all lazy-loaded.

If asked: explain the difference between authentication and authorization.

Slide 8 — API Integration

- One service per feature (TitlesService, ContractsService, …)
- Generic types: `HttpClient.get<Title[]>()`
- Envelope unwrap: `.pipe(map(r => r.data))`
- Interceptor adds JWT to every request

Mention error handling: status 0 / 401 / 5xx flows.

Slide 9 — Authentication Flow

- Login → backend returns JWT (text response).
- Client decodes payload → reads email + role.
- Stored in localStorage; restored on refresh.
- Interceptor sends Bearer header; 401 → auto-logout.

Mention XSS trade-off and HttpOnly cookie alternative.

Slide 10 — Demo Flow

- Login as ADMIN → Dashboard tiles render based on role.
- Open Titles → search + create → poster grid.
- Open Schedules → calendar → create → see conflicts page.
- Open Usage → date-range report.
- Open Users (ADMIN only) → create/delete user.

Run the actual app live if possible.

Slide 11 — Challenges

- Role-aware UI across 13 pages + 8 roles.
- Two different API envelope styles unified at service layer.
- Global 401 handling in interceptor.
- Converting Tailwind → plain CSS without changing UI.

Pick one to talk about in depth if asked.

Slide 12 — Future Enhancements

- Refresh-token flow for longer sessions.
- WebSocket for real-time conflict notifications.
- Server-Side Rendering (Angular Universal) for SEO of public pages.
- Light theme.
- Internationalisation (i18n).
- Unit + e2e test coverage (Karma / Cypress).

Shows you're aware of next steps and quality.

Slide 13 — Conclusion

- Built a production-grade Angular console for a microservices backend.
- Demonstrated modern Angular patterns: standalone, Signals, lazy loading, guards, interceptors.
- Followed industry best practices for security, architecture, and code organisation.
- Thank you — questions?

Stop, smile, invite questions. Keep this slide short.

14. Final Quick Revision (one-day notes)

14.1 Memorise this overnight

- **Entry:** `main.ts` → `bootstrapApplication(AppComponent, appConfig)`.
- **App config:** `provideRouter`, `provideHttpClient(withInterceptors([authInterceptor]))`, `provideAnimations`.
- **Standalone components** — no NgModule; each declares its own `imports`.
- **Lazy routes** — `loadComponent: () => import(...)`.
- **Guards** — functional, return `true|false`; use `inject()` inside.
- **Interceptor** — clone request, set `Authorization`, `catchError` on 401.
- **Signals** — `signal()` + `computed()` + (optional) `effect()`. Read with `()`.
- **Reactive forms** — `fb.nonNullable.group({ ... })`; validators in array.
- **HttpClient** — services return `Observable<T>`; component `subscribe`s.
- **RxJS** — `map` (transform), `tap` (side-effect), `catchError` (handle), `switchMap` (cancel-and-switch).
- **JWT** — base64 payload, exp in seconds, stored in localStorage; never trust it client-side.

14.2 Important interview points

1. "Standalone component" = no NgModule. Modern Angular default.
2. "Signals are sync, Observables are async streams" — different tools.
3. "OnPush + Signals" = best change-detection combo.
4. "Functional guards" use `inject()` instead of constructors.
5. "Interceptor handles auth and global errors in one place."
6. "Always wrap envelopes in the service" — components don't see `ApiResponse<T>`.
7. "Use trackBy in `*ngFor`" for performance.
8. "Lazy load every feature" — initial bundle stays small.

14.3 Frequently asked viva questions (1-liner answers)

Question	One-liner
Why Angular?	TS by default, batteries-included, opinionated, great tooling.
Why standalone components?	Less boilerplate, faster lazy loading, simpler DI.
What is DI?	Framework supplies dependencies instead of class constructing them.
What is an Observable?	A lazy stream you subscribe to.
What's RxJS for?	Compose async flows with operators.
JWT?	A signed token carrying claims used for stateless auth.
Lazy loading?	Load route code only when navigated to.
Reactive vs Template-driven?	Reactive in TS, scales better; template-driven simpler for tiny forms.
What is change detection?	Process Angular uses to update the DOM after state changes.
OnPush?	Strategy that re-checks only on input change or events — faster.
Async pipe?	Subscribes/unsubscribes to an Observable in a template.
Difference between trackBy and not?	trackBy reuses DOM nodes; without it, Angular destroys and recreates rows on data updates.
What is interpolation?	`{{ x }}` — print value in HTML.
Why interceptor?	Add headers / handle errors globally instead of per-call.
How is your project structured?	core/ for cross-cutting, layout/ for chrome, features/ per feature.

14.4 Short flashcards

Term	30-second definition
Component	UI unit = decorator + class + template + style
Service	Injectable class with logic/state
Router	Maps URL to component, supports lazy loading + guards
Guard	Function deciding if a route can activate
Interceptor	Middleware running on every HTTP request
Pipe	Template formatter
Directive	Modifies element behaviour or appearance
Signal	Reactive value holder (Angular 17)
Observable	Lazy async stream
RxJS	Library of operators for streams
DI	Framework supplies dependencies
JWT	Signed token = header.payload.signature
SPA	One HTML page, JS updates UI
CORS	Cross-origin request permission system
SSR	Server-side rendering for first paint

You're set. Open this PDF the night before any viva/interview. Read sections 1, 3, 8, 10.5, 14 thoroughly — they directly answer 90% of project-defence questions. Open sections 4 and 7 if asked anything code-specific. Good luck!